Lecture 03

CAP Theorem, Back of Envelope Calculation, Architecture

Just to know

Abtak hum jitna bhi application develop kar rahe the waha humara focus scalability pr tha.

Scalability kon provide kar raha tha? → Distributed System

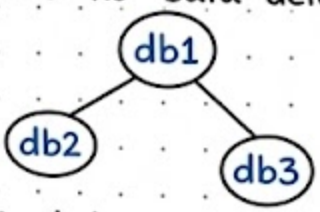
CAP Theorem

C : Consistency

A : Availability

P : Partition Tolerance

Suppose there are 3 db nodes, joh agar kuch changes hue toh baki dono ko bata deta hai



↳ Consistency : Client jab bhi data request kare toh use same data mile at same time, no matter on which node request has land

Example : Apna profile photo badla on Instagram, no matter koi apki profile kaha se dekh raha hai (i.e db node) sabko updated profile show hona

↳ Availability : When client sends request it should get a response, doesn't matter if it is stale data (i.e not updated data)

Example : Instagram khola toh data load hona chahiye it doesn't matter ki feed old hi hai

Partition Tolerance : Partition yaane suppose apka db jaha hai waha earthquake ya tsunami aq gyi, aur apka debasc db se connection tut gaya

↳ Breaking of application means partition

Aur apka joh application woh connection tootne ke baad kthi normal kaam kour raha hai that means, it has partition tolerance

↳ Inshert, our application should always be up or always operate.

- Statement of CAP Theorem : No matter what you will always get maximum of 2 out of these 3 i.e. CA or CP or AP

- Important point of Theorem : We will always need partition tolerance and consistency mandatory or availability are optional.

Why we cannot achieve all 3 at same time

→) Because, partitions are inevitable (Cannot be tolerated) because we need partition tolerance as our system is distributed system.

2) If our application cannot taterate partition then our system is centralized (one failure leads to application down).

3) Therefore you cannot have both during a partition

↳ Because, if one part of the system accepts a write & another part does too (without coordination), then nodes no longer agree.

⇒ Consistency lost

↳ If instead your force all partitions to agree before replying, then some partitions must wait (or reject requests) until the network heals ⇒ Availability lost

In One Line;

In a distributed system, when a network partition occurs, you must choose between Consistency (all nodes agree) and Availability (system keeps serving) YOU CAN NOT HAVE BOTH AT SAME TIME

Consistency vs Availability

↳ When to prefer, consistenty :: Response upto date honga.

e.g. Banking applications

↳ When to prefer availability : Response more important, hai doesn't matter if stale data hai

e.g. Instagram

Availability Numbers

- Har app ka apne users ke saath contract hota hai ki saal mein wah kitne din down ho sakti hai
  ↳ yaane no response
  - Koi application kabhi down nhi ho skti yaane 100% available hai
    Ex: Google, Amazon, Microsoft lies in 99.9 or 99.99%
    - Har company chahti hai decimal ke baad maximum 9's laana

* Real world application ka no agar 99 se kam hai like 98 so it is not good built application

{ Application are defined in number of 9's → Availability numbers }



If down time -

- 99% of 365 days : 3.65 days down
- 99.9% of 365 : 8.77 hours
- 99.99% of 365 : ~52.56 minutes

Back of the Envelope Calculations

This calculations helps in finding how better our application is through, QPS (Query Per, Second) and storage unit

↳ helps in improving response time

Helps in finding agar mein ek db le raha hu and wob hum approx 5 yrs use kreege toh maximum uska storage kya hona ek (estimate)

↳ means any CRUD operation / query on db

⊙ We will use example of instagram and perform Back of envelope calculations

Before Starting remember : power of 2^2

10 : Thousands	: Kilobyte
20 : Million	: megabyte
30 : Billion	: gigabyte
40 : Trillion	: terabyte
50 : Quadrillion	: petabyte
60 : exabyte	

Assumptions we need to consider

1. Monthly Active Users : ~ 2 Billion
   ↳ 60% of the users use Insta daily : ~ 1.2 Billion

   DAU : Daily Active User : 1.28

What users can do -

a) See feed (1 feed request = 1 Query)

b) User post photo/vedio/reel (1 post request = 1 Query)

2. Feed Check

↳ User check feed 30 ~~mint~~ times a day (avg)

$$\text{Feed View QPS} = \frac{1.2B * 30}{86400} = \sim 420\,K/sec$$

86400 ← 24hrs x 3600 sec

↱ on holidays/weekends

peek QPS = 2 * QP's
= 2 * 420 K/sec
= ~840 K/sec

3. Upload Check

↳ User publish 1 picture/reel everyday (avg)

QPS: Daily upload req = 1.2B * 1/86400 = ~14K/sec

peak QPS: = 2 * 14 K/sec = 28 K/sec

4. Storage Unit

Assumptions

↳ 20% of user → Video (~50 MB avg)

↳ 80% of user → picture (~1 MB avg)

- photos per day = 1.2B * 80% = ~1 B
- storage: 1 Billion * 1 MB = $2^{50} * 1 \times 2^{26} = 2^{50} =$ ~1 PB

(for storage, calculation always convert into power of 2)

Video per day = 1.2B * 20% = ~0.25 B

Storage: ~0.258 * 50 MB = $0.025 * 2^{30} * 50 * 2^{20} = 2^{50} \times 12 = 12$ PB

Total: 12 PB + 1 PB = 13 PB ⇒ per day

Calculation of 5 years :

13PB * 365 days * 5 years = ~24000 PB

↳ $24 \times 10^3 = 24 \times 2^{10} \times 2^{50} = 24 \times 2^{60} \times 2^{60}$

= 24 EB (exabyte)

Interview tip -

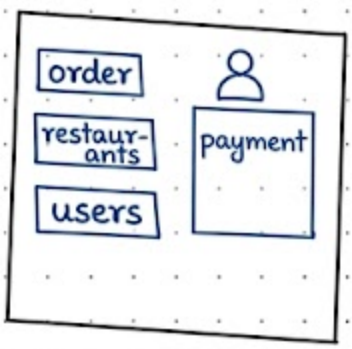
- Calculations karne padenge and îtna detail main hum batafenge to interviewer impress ho jayenge.
- Interview main aage hord, question nhi ayenge.

Monolith VS Microservices

- Monolith application ∴ saara logic ek hi repo-mein
  - ↳ har ek classes / logic / components of application in single file
  - ↳ this is legacy (bohot purane time se use

Advantage

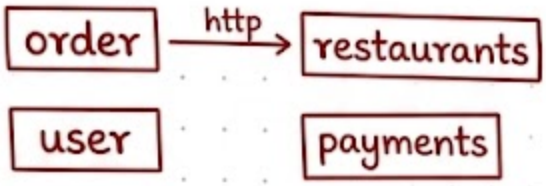
↳ Fast as communication b/w two modules are just method call or not http (calls over internet)



Disadvantages

→ Not much scalable [can't scale module]
→ Heavy application [CI/CD pipeline]
→ debug & testing hard

→ Microservices : deploy every logic/component in different services

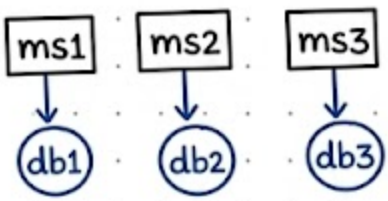


- Advantages :
  ↳ Scalable
  ↳ Light application
  ↳ easy debug & test

- Disadvantages :
  ↳ slow coz http call hoga over internet
  ↳ Transaction management
  ↳ Multiple microservice ko join karna.

— Let's discuss the disadvantage of microservices in detail

[] Transaction Management (ACID properties)



→ In short it means ek toh pura transaction hoga varna issue aagaya to role-back ho jaana

→ yeh monolith mein achieve karna easy hai cox ek hi service hai & uska ek hi db hai

→ Ab har ek ms (microservice) server ka apna db hoga toh woh apas mein communicate kaise karte ki successfull hai yaa nhi

e.g. ms1 par kuch kaam hua & success ho gaya operation par ms2 par error ke wajah se terminate ho gaya toh ab ms2 → ms1 ko kaise batayenga. tu rollback karle.

2) Multiple Microservices Join, →

→ yaha agar humne suppose ms2 ka code change kiya toh humme ms1 , ms3 ka bhi check karna padega , ki kya koi change required hai

→ Isse code mein multiple dependency aati hai

⊙ Ab in dono problem ka solutions hai unke patterns jisse hum phase of microservice ke baad discuss karenge.

Different phases of Microservices (IMP interview question)

- phases during conversion of monolith (me) to microservices (ms).

P-I] Decomposition → how will you breakdown your application (me → ms)

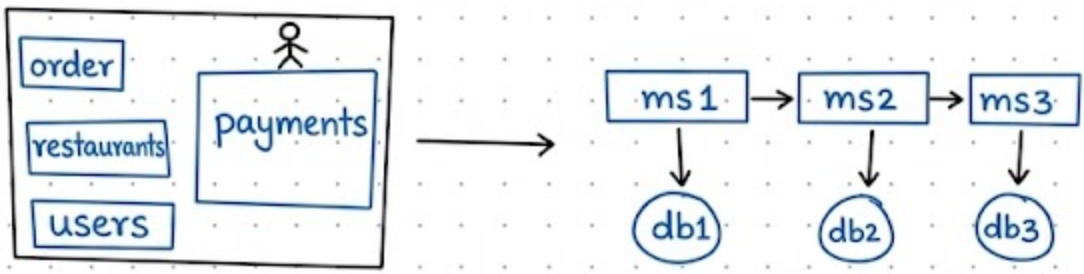
P-II] Database → Shared DB or unique DB for different ms. in application.

P-III] Communication → How ms will communicate : api / event driven system

P-V] Deployment → how will be deployed
P-II] Observability → how will we monitor
} handled by DevOps

I) Decomposition phase

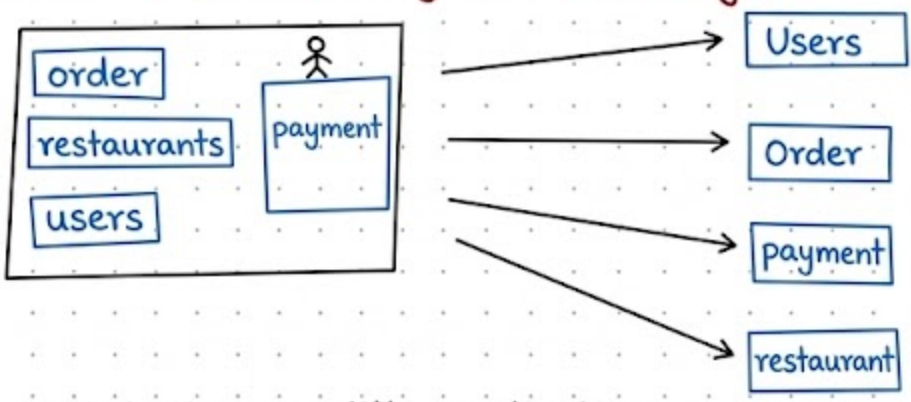
- If you have monolith → how will you break it to a microservice architecture



- Now we have design patterns to understand this things of how things will work

1) a) Decomposition by business logic
   b) Decemposition by subdomain
} yeh batayenge divide konse base par karma hai.

a) Decomposition by Business Logic



- divided the monolith application based on business logic
- yaane saare business components of Zamato divided in parts

Disadvantage

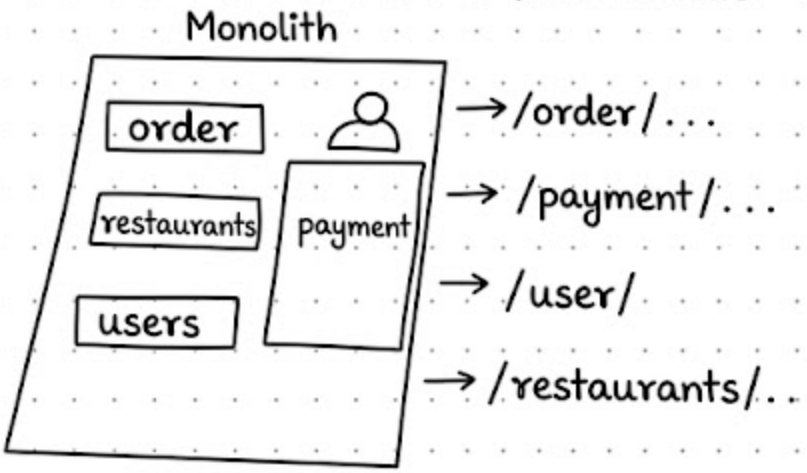
- developer should be aware of entire business.
- Confusion of keeping components in which section.

Ex: Agar "menu" component bhi aayja toh usse new entity /order / restaurants inmein se kaha rakhta.

b] Decomposition by Sub-domain

Humare applications mein already kuch domains bane honge toh uss hissab se hi divide karo. ~~gaise~~ kuch requests

/order → 1 entity (IMS)
/payment → 1 MS
/user → 1 MS
/restaurants → 1 MS

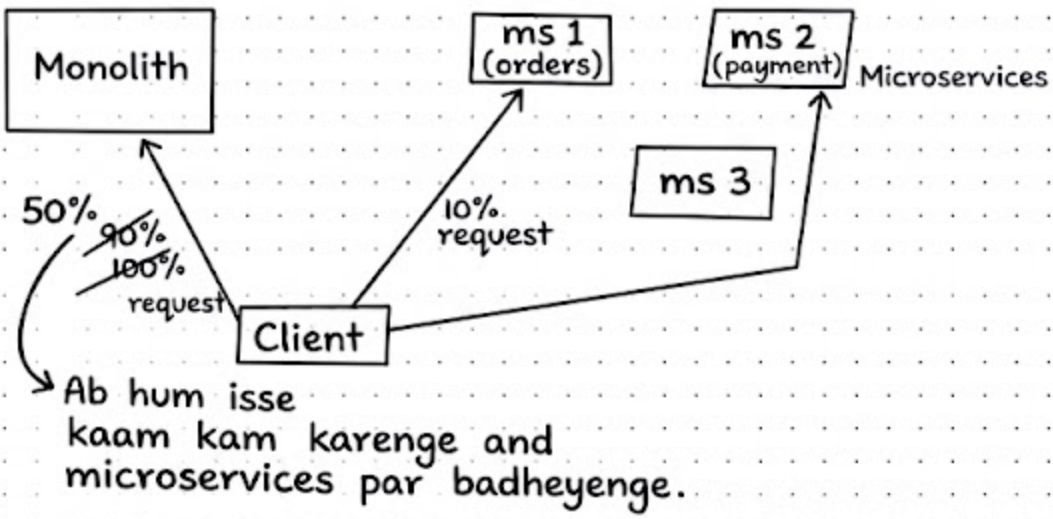


② Strangler pattern → Yeh batayega karonge kese (How will I do it?)

- Humne decide to toh krliya ki part kaise divide honge par monolith application deployed hai server par... toh microservices ko pehle deploy krdu & then shift traffic pr monolith saara handle nhi kar payega.

- Joh strangler pattern kehta hai ki hum humare monolith ko dheere dheere microservices mein convert kare

strangler means step by step



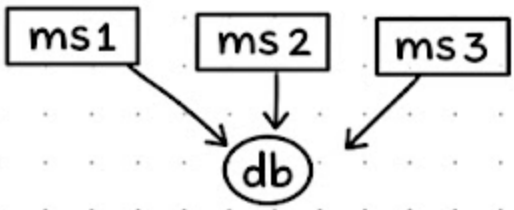
→ Strangler pattern kehta hai pehle ek module/ms build karlo (suppose order)

→ Ab 100% of api's request joh monolith ke server par jaati hai usko reduce karo to 90% & 10% ms pr bhejo

[Ab ise divide krne ke bhi 2 ways hai
1) by no. of apis (10-%, api on ms)
2) by no. of users (jaise 10-%, user on ms)

2) [Database Phase]
- Each microservice can have → shared db / unique db

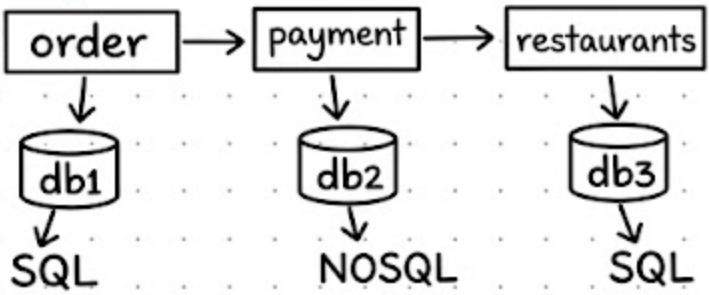
a) Shared DB



↳ Advantage
1) Simple to carry operation
2) JOINS (SQL) - easily
3) Transaction management (ACID)

↳ Disadvantage
1) Cannot be scaled properly
2) Limitation of either SQL / NOSQL (multiple options ✗)

b) Unique DB

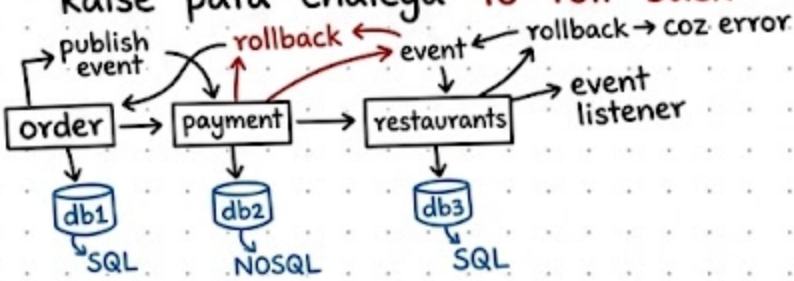


Disadvantages

1) Joins not possible (soln : CQRS)
2) Transaction management (soln : SAGA)

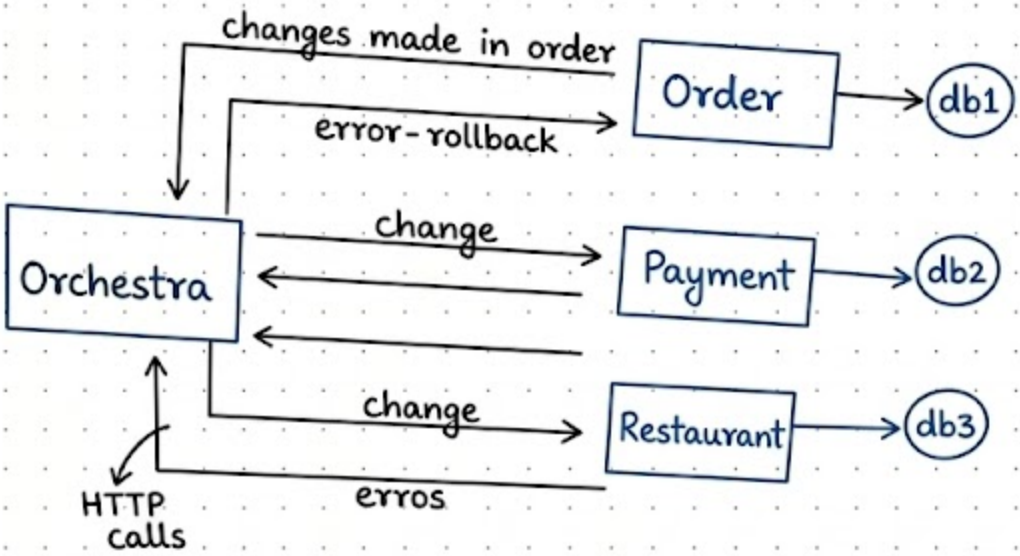
PS:
payments wala service directly, order ke database ko call nhi kar skta, order service ko bolna padega to call.

- SAGA Pattern
  ↳ soln of transaction mgmt problem - agar koi trxn fail ho gyi toh baakiyon ke kaise pata chalega to roll back



SAGA pattern states : jaise hi koi changes db mein ho toh ek event create karo & pass to next microservice. Agar trxn, successful hai toh forward for otherwise backward to ask them to rollback.

- Agar event driven system complicated lag rhi hai toh SAGA has has one more option → orchestration → orchestration



- In short it says, kuch bhi change hoga mujhe batao i will forward
- error aaya? i will ask others to rollback

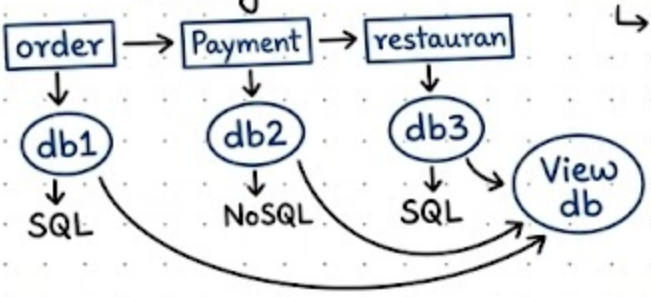
[cite: CQRS (Command Query Request Separation)

↳ soln of joins problem - different server par, deployed database par join operations kaise lagaye.

CQRS says:- Separate karo

↳ Command - write operation (update, delete, insert)

↳ Query - read (select)



↳ ek alag view db banao jismein saare tables present ho aur query ke result usmein